

Universidad CENFOTEC



Escuela de Software

Código del curso: SOFT-04.

Nombre del curso: Programación Orientada a Objetos.

Sección: SCV0.

Periodo: C1-2026.

Docente facilitador: Romario Salas Cerdas.

CONSIGNA DEL PRIMER AVANCE DE LA APLICACIÓN DE SOFTWARE

1. Datos generales de la actividad

Tipo de actividad:	Avance de proyecto.		
Fecha de entrega:	15 de marzo del 2026, 11:59pm.	Valor porcentual:	15%.
Formato de entrega:	Archivo .pdf , commit de código en repositorio de GitHub.	Puntaje total:	90.
Individual: No.	Grupal: Sí.		

2. Instrucciones generales

1. Lea cuidadosamente las instrucciones de la actividad. En caso de tener alguna duda, puede consultar con su docente.
2. Esta actividad se desarrolla de manera grupal. Cualquier intento de plagio será sancionado de acuerdo con el reglamento académico vigente.
3. Al completar la actividad, un representante del grupo debe realizar el commit del código a GitHub y subir un archivo **.pdf** a la plataforma Moodle en el tiempo y espacio indicado por el docente. El archivo **.pdf** debe contener el diagrama de clases UML. El repositorio debe incluir todo el código **.java**.

3. Objetivos o competencias del curso que se evaluarán en la actividad de aprendizaje

Objetivo general o competencia del curso	Diseña una aplicación de software con el paradigma orientado a objetos, usando relaciones entre objetos y clases, modelados de aplicaciones, bases de datos e interfaces.
Objetivos específicos que	• Crea un repositorio de la aplicación en una

se evalúan	plataforma de control de versiones en línea para tener una versión inicial del software. • Confecciona el diagrama de lenguaje unificado de modelado (UML), incluyendo las clases, métodos y atributos de los objetos. • Elabora una estructura básica del código en un lenguaje de alto nivel, que contiene los paquetes de la capa lógica y el diseño de interfaz. • Elabora el diagrama de lenguaje unificado final (UML), incluyendo asociación, agregación, composición y herencia. • Desarrolla un repositorio de la aplicación en la plataforma para la lógica de las clases, incluyendo los paquetes, lógica de negocio y entidades. • Verifica la funcionalidad de la aplicación, utilizando como mínimo los dos casos de prueba. • Realiza una explicación de la aplicación y contesta de forma correcta las preguntas propuestas por el docente. • Incorpora la retroalimentación que el docente brindó a los diferentes avances.
--------------------------	---

4. Descripción de la actividad

Contexto: Plataforma digital de subastas especializadas

Usted ha sido contratado para desarrollar una plataforma digital de intercambio entre pares, dirigida a una comunidad especializada de coleccionistas. La plataforma permite que los miembros de la comunidad ofrezcan objetos de valor y que otros usuarios interesados realicen ofertas durante un período de tiempo definido. A diferencia de plataformas generalistas, esta solución se enfoca en nichos específicos, donde la reputación, la confianza y el cumplimiento de las reglas son fundamentales para el funcionamiento del sistema. Una vez concluida una subasta, el objeto es adjudicado al mejor oferente y se inicia el proceso correspondiente para completar la transacción entre las partes. Usted es responsable del desarrollo completo de la aplicación, aplicando los principios de programación orientada a objetos, la arquitectura vista en clase y buenas prácticas de ingeniería de software.

Descripción general de la plataforma:

La plataforma permite administrar usuarios con distintos roles, los objetos ofrecidos dentro de la comunidad, las subastas activas y las subastas vencidas. Los usuarios interactúan en un entorno controlado, donde existen reglas claras sobre quién puede crear

subastas, quién puede realizar ofertas, cómo se determina el ganador de una subasta y cómo se registra la reputación de los participantes. El sistema debe garantizar la coherencia de la información y el estricto cumplimiento de las reglas de negocio definidas.

Dominio del sistema:

Objetos ofrecidos:

Un objeto representa un ítem de valor ofrecido en la plataforma. Cada objeto debe tener al menos:

- Nombre.
- Descripción.
- Estado (e.g., nuevo, usado, antiguo sin abrir).
- Fecha de compra.
- Antigüedad calculada (años, meses y días).

Usuarios del sistema:

La plataforma maneja tres tipos de usuarios: moderador, vendedor y coleccionista. Todos los usuarios deben tener al menos:

- Nombre completo.
- Identificación.
- Fecha de nacimiento.
- Edad calculada a partir de la fecha de nacimiento.
- Contraseña.
- Correo electrónico.

Adicionalmente:

- El vendedor y el coleccionista manejan una puntuación y una dirección.
- El coleccionista maneja una lista de intereses y una lista de objetos de su propiedad.

Subastas:

Una subasta representa la oferta de uno o más objetos en la plataforma. Cada subasta debe manejar al menos:

- Fecha de vencimiento.
- Tiempo restante para vencer (días, horas, minutos y segundos).
- Usuario creador de la subasta.
- Puntuación del creador.

- Precio mínimo de aceptación.
- Conjunto de objetos subastados.
- Estado de la subasta.

Ofertas:

Una oferta es una propuesta económica presentada por un coleccionista en una subasta.

Cada oferta debe manejar al menos:

- Nombre del oferente.
- Puntuación del oferente.
- Precio ofertado.

Orden de adjudicación:

Una orden representa la aceptación de la adjudicación de una subasta por parte del ganador. La orden debe manejar al menos:

- Nombre completo del ganador.
- Fecha de la orden.
- Detalle de los objetos adjudicados.
- Precio total de la orden.

Reglas de negocio:

1. Al iniciar la aplicación, el sistema debe verificar si existe un moderador registrado. Si no existe, se deben solicitar los datos para su registro.
2. Solo puede haber un único moderador en la plataforma.
3. El moderador no puede participar en subastas ni realizar ofertas.
4. El vendedor no puede realizar ofertas.
5. Si una subasta es creada por un coleccionista, dicho coleccionista no puede participar como oferente en ella.
6. No se puede crear una subasta si no tiene objetos asociados.
7. Para registrarse como vendedor o coleccionista se debe ser mayor de edad.
8. El moderador debe ser mayor de edad.
9. Si una subasta es creada por un coleccionista, este solo puede subastar objetos registrados en su colección.

Funcionalidad por desarrollar en el primer avance:

1. Archivo con el diagrama de clases siguiendo el estándar UML, que represente los atributos, los métodos y las relaciones de las siguientes clases: **Usuario**, **Subastas** y **Ofertas**.
2. Desarrollar la estructura básica de todas las clases indicadas en el punto anterior. Debe incluir encapsulamiento, al menos dos constructores (uno por defecto y otro que reciba los parámetros que asignen el valor a todos los atributos del objeto) y el método **toString()**.
3. Debe trabajar con la arquitectura vista en el curso: un proyecto aparte para la interfaz gráfica y otro para la arquitectura de la capa lógica. La estructura de paquetes debe reflejar dicha arquitectura. Recuerde que la interfaz gráfica no puede instanciar objetos de negocio.
4. El paquete raíz deberá ser **cr.ac.ucenfotec**.
5. La persistencia de datos debe lograrse mediante **ArrayLists** en memoria.
6. La interfaz gráfica debe ser un menú de consola, con las siguientes opciones:
 - Registro de usuarios.
 - Listado de usuarios.
 - Creación de subastas.
 - Listado de subastas.
 - Creación de ofertas.
 - Listado de ofertas.

Entregables:

1. Se debe incluir el código fuente completo de todas las clases desarrolladas.
2. Deben contar con un repositorio en GitHub para el proyecto de la interfaz gráfica y para el proyecto de la lógica de negocio.
3. Se debe incluir el diagrama UML de la capa lógica que refleje correctamente el diseño orientado a objetos implementado.
4. Se debe incluir Javadoc para todas las librerías generadas por el estudiante.
5. El proyecto debe implementar la arquitectura definida en el curso, respetando la separación entre capas.
6. Código legible y documentado (atributos privados, getters y setters, uso de **toString()** cuando aplique).

5. Rúbrica

Esta actividad de aprendizaje será evaluada mediante la siguiente rúbrica:

Criterio de evaluación	Insuficiente (1 punto)	Aceptable (2 puntos)	Bueno (3.5 puntos)	Excelente (5 puntos)
Modelado orientado a objetos y diagrama UML	El diagrama UML es inexistente o presenta errores graves de notación y diseño.	El diagrama incluye los elementos principales, pero presenta inconsistencias en relaciones o definición de métodos.	El diagrama representa correctamente las clases y la mayoría de las relaciones con notación adecuada.	El diagrama UML es claro, completo y correcto, representando fielmente el diseño orientado a objetos del sistema.
Encapsulamiento de atributos	Los atributos no están encapsulados o se manejan de forma incorrecta.	Se utiliza encapsulamiento de forma parcial o inconsistente.	Los atributos están correctamente encapsulados, con leves detalles por mejorar.	Todos los atributos están correctamente encapsulados, con uso consistente de modificadores de acceso y getters/setters.
Constructor por defecto	No se implementa el constructor por defecto o su implementación es incorrecta.	El constructor por defecto existe, pero presenta errores o uso limitado.	El constructor por defecto está correctamente implementado y funcional.	El constructor por defecto está correctamente implementado y se utiliza de forma coherente dentro del diseño.
Constructor general (con parámetros)	No se implementa el constructor general o no asigna correctamente los atributos.	El constructor general existe, pero no asigna correctamente todos los atributos.	El constructor general asigna correctamente la mayoría de los atributos.	El constructor general asigna correctamente todos los atributos y refleja un diseño claro y consistente.
Implementación del método toString()	No se implementa el método toString() o no	El método toString() existe, pero muestra	El método toString() muestra	El método toString() está correctamente

	cumple su propósito.	información incompleta o poco clara.	información relevante y bien estructurada.	implementado y facilita la visualización clara del estado del objeto.
Implementación de relaciones entre clases	Las relaciones no están implementadas o no coinciden con el modelo definido.	Las relaciones existen, pero presentan inconsistencias con el diseño UML.	Las relaciones están correctamente implementadas en su mayoría.	Todas las relaciones están correctamente implementadas y reflejan fielmente el diseño UML.
Arquitectura por capas	No se respeta la arquitectura definida y se mezclan responsabilidades entre capas.	Se intenta aplicar la arquitectura, pero existen errores en la separación de responsabilidades.	La arquitectura por capas está bien aplicada, con pequeños detalles por mejorar.	La arquitectura por capas se implementa correctamente, con separación clara y estricta entre capas.
Instanciación de objetos de negocio en la interfaz gráfica (UI)	La interfaz gráfica instancia directamente objetos de negocio de forma reiterada y sin respetar la arquitectura definida.	La interfaz gráfica instancia algunos objetos de negocio, aunque existe un intento parcial de delegar responsabilidades.	La interfaz gráfica instancia mínimamente objetos de negocio, existiendo pocos casos incorrectos.	La interfaz gráfica no instancia objetos de negocio; toda la creación y gestión de objetos se realiza exclusivamente en la capa lógica, respetando estrictamente la arquitectura definida.
Estructura de paquetes	La estructura de paquetes es incorrecta o no refleja la arquitectura definida.	La estructura de paquetes existe, pero presenta inconsistencias o desorden.	La estructura de paquetes es clara y en su mayoría coherente con la arquitectura.	La estructura de paquetes es clara, ordenada y refleja correctamente la arquitectura del sistema.
Registro de usuarios (menú)	La opción no funciona o no	Permite registrar usuarios, pero con	Permite registrar usuarios	Permite registrar usuarios

	permite registrar usuarios correctamente.	errores de validación o flujo.	correctamente en la mayoría de los casos.	correctamente, aplicando validaciones y reglas de forma consistente.
Listado de usuarios (menú)	No se listan los usuarios o la información es incorrecta.	Se listan los usuarios, pero con información incompleta o desordenada.	Se listan correctamente los usuarios con información relevante.	El listado de usuarios es claro, completo y bien presentado.
Creación de subastas (menú)	No permite crear subastas o incumple reglas básicas.	Permite crear subastas, pero no valida correctamente todas las reglas.	Permite crear subastas aplicando la mayoría de las reglas del sistema.	Permite crear subastas aplicando correctamente todas las reglas definidas.
Listado de subastas (menú)	No se listan las subastas o la información es incorrecta.	Se listan las subastas, pero con información incompleta o poco clara.	Se listan correctamente las subastas con información relevante.	El listado de subastas es claro, completo y refleja correctamente el estado del sistema.
Creación de ofertas (menú)	No permite crear ofertas o incumple reglas básicas del sistema.	Permite crear ofertas, pero no valida correctamente todas las restricciones.	Permite crear ofertas aplicando la mayoría de las reglas del sistema.	Permite crear ofertas aplicando correctamente todas las reglas definidas.
Listado de ofertas (menú)	No se listan las ofertas o la información es incorrecta.	Se listan las ofertas, pero con información incompleta o desordenada.	Se listan correctamente las ofertas con información relevante.	El listado de ofertas es claro, completo y consistente con el estado del sistema.
Persistencia en memoria con `ArrayList`	No se implementa correctamente la persistencia en memoria.	Se utiliza persistencia en memoria, pero con uso limitado o incorrecto de las	Se implementa correctamente la persistencia en memoria utilizando ArrayList , con	La persistencia en memoria está correctamente implementada, con uso claro, ordenado y

		estructuras de datos.	manejo adecuado de los datos.	consistente de ArrayList .
Calidad, legibilidad y documentación del código	Código desordenado, poco legible y sin documentación.	Código funcional, pero con problemas de legibilidad o documentación incompleta.	Código legible y bien documentado, con buenas prácticas básicas.	Código claro, ordenado y completamente documentado, incluyendo Javadoc y buenas prácticas.
Cumplimiento de entregables y reglas de la consigna	No cumple con los entregables mínimos o el proyecto no compila/ejecuta.	Cumple parcialmente con los entregables solicitados.	Cumple con la mayoría de los entregables, formatos, restricciones y reglas establecidas.	Cumple completamente con todos los entregables, formatos, restricciones y reglas establecidas en la consigna.